

HAProxy / PgBouncer — Operator Manual

What these technologies do

HAProxy gives clients stable endpoints while backend roles change. PgBouncer pools PostgreSQL client connections and enforces the commissioned database authentication/TLS boundary.

For database traffic, the current path is:

```
client
-> PgBouncer :6432
-> HAProxy PostgreSQL-primary route :15432
-> current Patroni leader :5432
```

The same HAProxy installations also serve other approved private/public routes. Do not edit one frontend without validating the whole HAProxy configuration.

Current database listeners

- PgBouncer client endpoint: :6432 on each ULC host.
- HAProxy writable-primary route: :15432.
- HAProxy replica/test route: :15433 where documented.
- Backend PostgreSQL members: private :5432.

Step 1 — Prove Patroni first

Before blaming the routers, establish one database leader:

```
sudo docker exec -e LC_ALL=C spilo patronictl -c /run/postgres.yml list
```

If Patroni has no safe leader, stop. Fix etcd/Patroni before changing HAProxy/PgBouncer.

Step 2 — Check service state

```
sudo systemctl status haproxy --no-pager -l
sudo systemctl status pgbouncer --no-pager -l
```

A running service is not enough. Continue to listeners and real SQL.

Step 3 — Validate HAProxy configuration before reload/restart

```
sudo haproxy -c -V -f /etc/haproxy/haproxy.cfg
```

PASS means: configuration validates with no fatal error.

Never reload an invalid HAProxy config. Always run this command after editing and before `systemctl reload haproxy`.

Step 4 — Check database-routing listeners

```
sudo ss -lntp | grep -E ':(5432|6432|15432|15433)\b' || true
```

Check that intended listeners are on the expected private/local address, not unexpectedly on `0.0.0.0/public` interfaces.

Step 5 — Inspect PgBouncer health through its admin interface

Using the protected PgBouncer admin/stats connection already provisioned for this host, run:

```
SHOW POOLS;  
SHOW SERVERS;  
SHOW STATS;
```

Look for:

- excessive waiting clients;
- no available server connections;
- repeated auth failures;
- server target inconsistent with the current primary route.

Do not copy passwords from `/etc/pgbouncer/userlist.txt` into documentation or chat.

Step 6 — Prove a real SQL connection through the normal path

Using a protected least-privilege DSN for `pgbouncer.internal.lorawan.com:6432`, run:

```
SELECT inet_server_addr(), pg_is_in_recovery();
```

PASS means: TLS/auth succeeds and a primary-route client returns `pg_is_in_recovery() = false`.

If PgBouncer accepts the client but the SQL backend fails, inspect HAProxy backend/Patroni/TLS. If direct HAProxy works but PgBouncer fails, inspect PgBouncer auth/pool/client TLS.

Step 7 — Use layer-by-layer isolation

Test in this order:

```
client -> :6432 listener  
client auth/TLS -> PgBouncer  
PgBouncer -> :15432  
HAProxy -> Patroni primary backend  
PostgreSQL TLS/HBA/role  
SQL operation
```

Do not change PostgreSQL passwords because HAProxy has no healthy backend, and do not change HAProxy because a client supplied the wrong PgBouncer credential.

Safe HAProxy change procedure

1. copy `/etc/haproxy/haproxy.cfg` to a protected rollback file;
2. edit only the intended frontend/backend;
3. run `haproxy -c -V -f /etc/haproxy/haproxy.cfg`;
4. if validation fails, do not reload;
5. reload rather than restart when a reload is sufficient;
6. confirm listeners;
7. confirm backend-specific health;
8. run one real client operation.

Command after a successful validation:

```
sudo systemctl reload haproxy
sudo systemctl is-active haproxy
```

Safe PgBouncer change procedure

1. prove Patroni/HAProxy path works first;
2. back up `pgbouncer.ini` and the protected `userlist/auth` source;
3. change only the intended database/role/pool/TLS item;
4. validate permissions and certificate paths;
5. reload/restart according to the setting changed;
6. run `SHOW POOLS`, `SHOW SERVERS`, and one real SQL query;
7. verify application reconnect behavior.

Do not broaden a database role because one client was configured with the wrong credentials.

PostgreSQL role-auth synchronization rule

PgBouncer's commissioned SCRAM `userlist` must match the approved PostgreSQL application/evidence roles. When adding/rotating a role, follow the database/PgBouncer rotation procedure and prove the new role can do only its intended operations.

Do not copy the whole password/userlist file to another environment as a shortcut.

Troubleshooting map

Symptom	Likely layer
connection refused on 6432	PgBouncer service/listener
client TLS hostname error	PgBouncer certificate/SAN/client hostname
authentication failed at 6432	PgBouncer userlist/role/credential
PgBouncer has no server	HAProxy route/backend or upstream TLS
HAProxy primary route unhealthy	Patroni leader/REST/backend reachability

Symptom	Likely layer
SQL reaches replica on primary route	HAProxy Patroni health selection defect
only one app fails	that app's role/DSN/pool, not necessarily cluster

Recovery rules

Do not bypass PgBouncer/HAProxy permanently during an incident. Direct-member connections are diagnostic/admin tools; restore the stable client path before declaring recovery complete.

After a PostgreSQL leader change, applications should reconnect without DSN edits. If manual endpoint edits are required, the HA routing layer is not working as intended.

Completion checklist

- Patroni has one leader;
- HAProxy config validates;
- HAProxy and PgBouncer active;
- expected listeners present/private;
- PgBouncer pools/servers sane;
- a real SQL query through :6432 reaches a writable primary;
- app roles authenticate with least privilege;
- no application is pinned to a current member IP.

Detailed runbook

- [../server/cloud-production/07-haproxy-and-pgbouncer.md](#)

Supporting runbook: 07-haproxy-and-pgbouncer.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

7. PgBouncer and HAProxy Core Service Routing

Status: CORE + EVIDENCE AUTH COMPLETE / PASS. HAProxy 2.8.16-0ubuntu0.24.04.3 and PgBouncer 1.22.0-1build4 are commissioned on all three nodes. The stable database client path is client TLS -> local PgBouncer :6432 -> local HAProxy primary route :15432 -> current Patroni leader over verified PostgreSQL TLS. The original four-role SCRAM/failover boundary remains historical PASS, and all three nodes now carry the same authoritative ten-entry/six-evidence SCRAM userlist SHA-256 665f1592c96ca276681a454b9cbcd6ab8ab0cbfb4594b8ddd443239db58df391. Fresh strict verify-full evidence sessions through all three physical PgBouncer endpoints passed. Historical Phase 7 sections that say “exactly four” describe the original commissioning baseline; this current header and the newest evidence-expansion checkpoint supersede them for live userlist count.

7.1 Purpose

The tiny HA POC runs PgBouncer and the private PostgreSQL HAProxy frontend on **all three hosts**. ha-01 and ha-02 are the public-ingress candidates. The original target design reserved ha-03:18883 for Node-RED, but live Phase 9 uses :18883 -> :8885 only on ulc-01/02 for ChirpStack. The refined Phase 12A design instead commissions a separate Node-RED mTLS frontend on ulc-03:18884 -> Mosquitto :8886; treat that later route as authoritative for Node-RED.

```
local database client
-> pgbouncer.internal.lorawan.com:6432
-> PgBouncer on this host
-> postgres-ha.internal:15432
-> HAProxy on this host
-> current Patroni primary on private TCP/5432

Clients include:
ha-01 ChirpStack-1 + Fabric adapter-1
ha-02 ChirpStack-2 + Fabric adapter-2
ha-03 Node-RED + Grafana
```

6432 is the stable local pool endpoint. 15432 is the local HAProxy PostgreSQL-primary endpoint and avoids colliding with PostgreSQL 5432 on the same Droplet. Map the logical service names to the current host's private VPC IP for each local client/container.

This gives every database client the same failover shape without another database router. ha-03 is not a public Reserved-IP candidate; its HAProxy instance is reused for private database routing and, when Phase 12A is activated, the dedicated Node-RED :18884 MQTT route described in [12a-node-red-timescale-telemetry.md](#).

HAProxy on ha-01 and ha-02 also provides the stable private OpenBao KMS route used by the two Fabric adapter workers:

```
Fabric adapter on this app host
-> openbao-kms.internal.lorawan.com:18200
-> HAProxy on this app host
-> one initialized, unsealed OpenBao-1/2/3 API backend on private TCP/8200
```

The KMS frontend stays in TCP mode so application TLS remains end-to-end to OpenBao. OpenBao Integrated Storage handles active/standby behavior and request forwarding inside the three-member cluster; HAProxy's job is only to keep one stable client endpoint and avoid routing to an unusable backend.

7.2 Preconditions

- Patroni cluster has one primary and healthy replicas.
- Patroni REST endpoints are reachable from all three hosts on private port 8008.
- PostgreSQL TLS certificates and CA are installed.
- HAProxy and PgBouncer versions are pinned and supported.
- ChirpStack database role exists.
- Maximum PostgreSQL connections and application concurrency are approved.
- Before enabling the KMS frontend, OpenBao-1/2/3 exist as one initialized Integrated Storage/Raft cluster, all intended usable members are unsealed, and their API certificates validate the approved private KMS identity.

7.2.1 Activation preflight - run before package installation

Run this same read-only gate once on ulc-01, ulc-02, and ulc-03. Do not run apt update, install packages, create certificates, or start services yet. Capture all three outputs first.

```
sudo -v

bash -s <<'EOF'
set -euo pipefail
set +x

NODE="$(hostname)"

case "$NODE" in
    ulc-01) NODE_IP='10.104.0.2' ;;
    ulc-02) NODE_IP='10.104.0.4' ;;
    ulc-03) NODE_IP='10.104.0.8' ;;
    *) echo 'FAIL: run only on ulc-01, ulc-02, or ulc-03'; exit 1 ;;
esac

echo "=== $NODE PHASE 7 HAPROXY/PGBouncer READ-ONLY PREFLIGHT ==="

echo
echo '=== 1. POSTGRES/PATRONI BASELINE ==='
for SPEC in \
    'ulc-02|10.104.0.4|leader' \
    'ulc-01|10.104.0.2|replica' \
    'ulc-03|10.104.0.8|replica'
do
    IFS='|' read -r CHECK_NODE IP ENDPOINT <<< "$SPEC"
    HTTP=$(curl -s -o /dev/null -w '%{http_code}' --max-time 5 \
        "http://$IP:8008/$ENDPOINT" || true)
    echo "$CHECK_NODE /$ENDPOINT = HTTP $HTTP"
    [ "$HTTP" = '200' ] || { echo 'FAIL: Patroni baseline changed'; exit 1; }
done

sudo docker exec -e LC_ALL=C spilo \
    patronictl -c /run/postgres.yml list

echo 'Patroni baseline: PASS'
```

```

echo
echo '=== 2. CURRENT HOST RESOURCE SNAPSHOT ==='
printf 'host = %s\n' "$NODE"
printf 'expected VPC IP = %s\n' "$NODE_IP"
ip -br address show
free -m
df -h /

echo
echo '=== 3. PACKAGE STATE - NO INSTALL ==='
for PKG in haproxy pgbouncer; do
    INSTALLED=$(dpkg-query -W -f='${Status}|${Version}' "$PKG" 2>/dev/null || true)
    if [ -n "$INSTALLED" ]; then
        echo "$PKG installed = $INSTALLED"
    else
        echo "$PKG installed = absent"
    fi

    echo "--- apt policy: $PKG ---"
    apt-cache policy "$PKG" | sed -n '1,12p'
done

echo
echo '=== 4. SERVICE UNIT STATE ==='
for UNIT in haproxy.service pgbouncer.service; do
    LOAD=$(systemctl show "$UNIT" -p LoadState --value 2>/dev/null || true)
    ACTIVE=$(systemctl is-active "$UNIT" 2>/dev/null || true)
    ENABLED=$(systemctl is-enabled "$UNIT" 2>/dev/null || true)
    echo "$UNIT load=${LOAD:-not-found} active=${ACTIVE:-unknown} enabled=${ENABLED:-unknown}"
done

echo
echo '=== 5. PLANNED DATABASE ROUTING PORTS MUST BE FREE ==='
COLLISIONS=$(sudo ss -ltnpH | awk '
    $4 ~ /:6432$/ ||
    $4 ~ /:15432$/ ||
    $4 ~ /:15433$/ { print }
')

if [ -n "$COLLISIONS" ]; then
    printf '%s\n' "$COLLISIONS"
    echo 'FAIL: one or more Phase 7 database-routing ports are already occupied'
    exit 1
fi

echo '6432, 15432, 15433 listener collision check: PASS'

echo
echo '=== 6. POSTGRES CA MATERIAL ==='
PG_CA='/etc/lorawan-pki/postgres/ca.crt'

# /etc/lorawan-pki/postgres is intentionally restrictive. Use privileged
# traversal so an operator outside the service group does not get a false
# "missing" result from a plain shell [ -f ] test.
if ! sudo test -f "$PG_CA"; then
    echo "FAIL: privileged check cannot find $PG_CA"
    exit 1
fi

sudo stat -c '%a|%u|%g|%n' "$PG_CA"
sudo openssl x509 -in "$PG_CA" -noout -subject -issuer -fingerprint -sha256
echo 'PostgreSQL CA: PASS'

echo
echo '=== 7. PGBOUNCER CLIENT-TLS MATERIAL INVENTORY ==='
for FILE in \
    /etc/lorawan-pki/pgbouncer/ca.crt \
    /etc/lorawan-pki/pgbouncer/server.crt \
    /etc/lorawan-pki/pgbouncer/server.key
do
    if sudo test -e "$FILE"; then
        sudo stat -c 'present|%a|%u|%g|%n' "$FILE"
    else

```

```

    echo "not-provisioned|$FILE"
  fi
done

echo
echo '=== 8. EXISTING CONFIG INVENTORY ==='
for FILE in \
  /etc/haproxy/haproxy.cfg \
  /etc/pgbouncer/pgbouncer.ini \
  /etc/pgbouncer/userlist.txt
do
  if sudo test -e "$FILE"; then
    sudo stat -c 'present|%a|%u|%g|%n' "$FILE"
  else
    echo "absent|$FILE"
  fi
done

echo
echo "$NODE PHASE 7 READ-ONLY PREFLIGHT: PASS"
EOF

RC=$?
echo
echo "Child gate exit code = $RC"
echo 'SSH LOGIN SHELL IS STILL ALIVE'

```

Why: Phase 7 introduces three new private listeners on every host. Checking the ports before installation separates a real collision from a later HAProxy/PgBouncer configuration problem. Capturing apt-cache policy before installation records exactly what Ubuntu would install instead of silently assuming a version. The PostgreSQL CA is already required for PgBouncer's verified server connection. PgBouncer's own client-facing certificate bundle may still be not-provisioned; record that state rather than loosening client_tls_sslmode or hostname verification as a shortcut.

Recorded u1c-03 HAProxy canary harness attempt - 2026-08-23: INCONCLUSIVE / NO MUTATION. The database HA baseline passed, then the script exited with code 141 at the package-candidate extraction step before apt-get install was reached. Root cause is shell behavior: with set -o pipefail, apt-cache policy haproxy | awk '/Candidate:/ {print \$2; exit}' lets awk close the pipe as soon as it finds the candidate; apt-cache then receives SIGPIPE and returns 141, making the command substitution fail. This is not evidence against HAProxy 2.8.16-0ubuntu0.24.04.3. Use a parser that consumes the full stream, such as sed -n 's/^ Candidate: //p', and avoid grep -q pipelines where an upstream writer could receive SIGPIPE. Because the failure occurred before installation, no HAProxy package, service, listener, or config mutation was made.

The preflight is PASS when Patroni stays healthy, the three planned database-routing ports are free, and the PostgreSQL CA exists. Package/config/PgBouncer-certificate lines are inventory evidence and can legitimately report absent or not-provisioned at this point. Review all three hosts before installing anything.

7.2.2 Recorded u1c-03 HAProxy database-routing canary - 2026-08-23

PASS. The corrected retry first re-proved the Patroni baseline, the pinned HAProxy package candidate, package absence, and free 15432/15433 ports. apt-get -s showed only haproxy 2.8.16-0ubuntu0.24.04.3 plus liblua5.4-0, with no removals. The exact HAProxy version was then installed. The package-default configuration was retained at /etc/haproxy/phase7-canary-20260823-141310/haproxy.cfg.package-default before the canary configuration replaced /etc/haproxy/haproxy.cfg.

The canary configuration passed offline and installed syntax validation. Ubuntu's HAProxy package post-install automatically created the systemd enablement symlink; the gate then stopped the package-default service before replacing the configuration, started the validated private-only canary, and explicitly confirmed enablement again after runtime checks. ss showed exactly 10.104.0.8:15432 and 10.104.0.8:15433, with no wildcard/public database-routing listener. A verify-full PostgreSQL session through 10.104.0.8:15432 landed on the current primary as 10.104.0.4|10.104.0.8|f|t|TLSv1.3|TLS_AES_256_GCM_SHA384. The replica frontend 10.104.0.8:15433 landed

on ulc-01 as 10.104.0.2|10.104.0.8|t|t|TLSv1.3|TLS_AES_256_GCM_SHA384. Patroni remained ulc-02 leader with ulc-01 and ulc-03 streaming replicas and all expected role endpoints HTTP 200. Child gate exit code was 0. Future node rollouts should explicitly disable --now haproxy immediately after package installation so boot persistence is intentionally restored only after routing validation.

Why this matters: it proves the HAProxy-only database-routing design independently from PgBouncer. PostgreSQL TLS stays end-to-end through HAProxy, Patroni REST decides which backend is eligible, and the local HAProxy source address matches the hardened /32 PostgreSQL HBA rules already commissioned. No PostgreSQL/Patroni configuration changed, and PgBouncer was not installed or started.

Next: repeat this exact database-routing behavior on ulc-01; only after the second replica-host rollout passes should ulc-02 receive HAProxy.

7.2.3 Recorded ulc-01 HAProxy database-routing rollout - 2026-08-23

PASS. The second-host rollout re-proved the expected Patroni topology (ulc-02 /leader, ulc-01 /replica, ulc-03 /replica, all HTTP 200), pinned candidate haproxy 2.8.16-0ubuntu0.24.04.3, package absence, and free 15432/15433 ports. apt-get -s again showed only liblua5.4-0 and the pinned HAProxy package, with no removals. After installation, the package-created systemd enablement was deliberately neutralized with systemctl disable --now haproxy; observed state was active=inactive, enabled=disabled before any custom listener was started. The package-default configuration was backed up at /etc/haproxy/phase7-rollout-20260823-142347/haproxy.cfg.package-default.

The node-specific configuration passed both off-path and installed syntax checks. HAProxy then started in runtime-only state while remaining disabled. ss showed exactly 10.104.0.2:15432 and 10.104.0.2:15433. A verify-full PostgreSQL session through the primary frontend returned

10.104.0.4|10.104.0.2|f|t|TLSv1.3|TLS_AES_256_GCM_SHA384, proving traffic landed on current leader ulc-02 while PostgreSQL saw HAProxy source 10.104.0.2. The replica frontend returned

10.104.0.2|10.104.0.2|t|t|TLSv1.3|TLS_AES_256_GCM_SHA384, which is valid because ulc-01 is itself a healthy replica and may be selected locally for replica traffic. Patroni remained healthy with both replicas streaming and reported lag 0. Only after these runtime checks did the gate explicitly enable HAProxy for boot persistence. Child gate exit code was 0; PgBouncer remained uninstalled and PostgreSQL/Patroni configuration was unchanged.

Why this matters: HAProxy database routing is now proven on two independent replica-side hosts, including the /32 source-address behavior required by the hardened HBA. The final rollout target is ulc-02, the current PostgreSQL leader; its rollout must prove installing HAProxy does not change PostgreSQL leadership, Patroni state, or Spilo container identity.

7.2.4 Recorded ulc-02 current-leader HAProxy rollout - 2026-08-23

PASS. Before mutation, ulc-02 /leader, ulc-01 /replica, and ulc-03 /replica all returned HTTP 200; local pg_is_in_recovery() was f; and the Spilo container identity was 2313de94ee2c5dcc292ac28ea7ae8359fbe31b1ea6ab171276306716cff58762|2026-08-22T14:19:16.323317366Z|0. HAProxy was absent, 15432/15433 were free, the candidate was the pinned 2.8.16-0ubuntu0.24.04.3, and the apt simulation showed only liblua5.4-0 plus HAProxy with no removals.

After exact-version installation, the package-created systemd state was deliberately neutralized to active=inactive, enabled=disabled. PostgreSQL was immediately rechecked and remained leader with pg_is_in_recovery()=f. The package-default config was preserved at /etc/haproxy/phase7-rollout-20260823-143511/haproxy.cfg.package-default. The node-specific config passed off-path and installed syntax checks, then HAProxy started while still disabled. Exact listeners were only 10.104.0.4:15432 and 10.104.0.4:15433.

Primary-route verify-full evidence was 10.104.0.4|10.104.0.4|f|t|TLSv1.3|TLS_AES_256_GCM_SHA384, proving the local HAProxy primary frontend correctly selected the local current leader. Replica-route evidence was 10.104.0.2|10.104.0.4|t|t|TLSv1.3|TLS_AES_256_GCM_SHA384, proving the replica frontend selected a healthy

standby while PostgreSQL saw the hardened /32 HAProxy source 10.104.0.4. The Spilo container identity after rollout was byte-for-byte identical to the pre-rollout identity. Final Patroni state remained u1c-02 Leader/running on timeline 2 with u1c-01 and u1c-03 streaming and reported lag 0. Only after all runtime checks passed was HAProxy enabled. Child gate exit code was 0; PgBouncer remained uninstalled and PostgreSQL/Patroni configuration was unchanged.

Three-node HAProxy database-routing boundary: PASS. Do not add another Patroni switchover merely to prove HAProxy yet. The permanent PgBouncer verification name is `pgbouncer.internal.lorawan.com`.

PgBouncer CA read-only inventory - 2026-08-23: PASS. On u1c-03, `/root/lorawan-pg-ca` remained mode 0700 root:root. The commissioned CA certificate is exactly `/root/lorawan-pg-ca/ca.crt`, subject/issuer CN = LoRaWAN PostgreSQL Internal CA, valid 2026-08-22 through 2036-08-19, with SHA-256 fingerprint 99:00:4B:B3:2D:7D:78:FA:38:61:7C:78:89:6D:7A:7E:FF:9F:A6:10:FC:8F:07:D4:E2:5E:35:25:36:E6:CB:3E. Its certificate public-key SHA-256 is 2da3b7630e38a1e80469c4d5d1f1c4ac9f1125fb47d6c5a4c2d411960340a2f9, and only `/root/lorawan-pg-ca/ca.key` matched that public key. Existing PostgreSQL member keys did not match, as expected. The inventory printed no private-key contents and made no filesystem mutation.

First PgBouncer issuance harness attempt - 2026-08-23: INCONCLUSIVE / NO CERTIFICATE MUTATION. The gate re-proved the CA directory, CA certificate, CA key permissions, commissioned CA fingerprint, and that the CA certificate and CA private key both derive the same public-key SHA-256 2da3b7630e38a1e80469c4d5d1f1c4ac9f1125fb47d6c5a4c2d411960340a2f9. The harness then failed before staging-directory creation because its hard-coded expected hash accidentally omitted the final 9, so the correct observed 64-hex hash did not equal the typoed expectation. Child exit code was 1. No staging directory, node key, CSR, certificate, PgBouncer package, HAProxy change, or PostgreSQL/Patroni change was reached. Correct the expected hash and rerun the same issuance design.

Corrected PgBouncer three-node TLS issuance - 2026-08-23: PASS. The corrected gate used the observed CA public-key SHA-256 ending in `...0a2f9`, re-proved PgBouncer was still absent, hashed the existing `ca.sr1`, and created root-only staging directory `/root/lorawan-pg-ca/pgbouncer-issuance-20260823-150532`. It generated one unique RSA-3072 key and CSR per node, signed each certificate with the existing CN = LoRaWAN PostgreSQL Internal CA for 825 days using a random 128-bit serial, and applied `serverAuth` plus exact SANs for `pgbouncer.internal.lorawan.com`, the node hostname, and node VPC IP.

Issued identities:

- u1c-01 / 10.104.0.2: serial 7C5887FBE0338797CAAC8230AD7D89F8, certificate fingerprint A4:EC:DF:86:30:68:29:88:0F:52:05:0A:E1:B7:E5:F9:3E:B3:4A:72:72:81:96:40:BC:10:7A:EC:94:D2:6D:E1, key/certificate public-key SHA-256 ba76dd9cde0722cb1377446837cf6f29fffc38550b42a63667c9f2fd8787fc5.
- u1c-02 / 10.104.0.4: serial 16F56AB3A41FF77DB93EE38EE377164D, certificate fingerprint 3B:25:17:30:2B:FB:26:7D:49:F7:C5:24:C5:B0:47:F6:BF:D1:88:64:8D:FD:0E:05:9B:ED:08:32:A1:50:02:EA, key/certificate public-key SHA-256 3dcd2400c7ed3936aa3fb5aa0360c4e3c366153d597edbc20ec6243ec09c68cf.
- u1c-03 / 10.104.0.8: serial 058F982D16D50B7AE8FF266ACFFBCBBE, certificate fingerprint BC:50:50:8D:FC:60:37:5E:E8:B6:A0:3B:93:41:D7:AB:53:CB:BD:C7:E1:86:3B:1D:FD:AD:D8:94:9E:45:7F:6E, key/certificate public-key SHA-256 4784b3d7c22993eb5e5577b136962a29902d504f59aae71cc2c9de733ab0a716.

Every certificate passed CA-chain verification, `-purpose sslserver`, `-verify_hostname` `pgbouncer.internal.lorawan.com`, node-hostname verification, node-IP verification, and private-key/certificate public-key equality. Unique key, certificate-fingerprint, and serial counts were all 3. Existing `ca.sr1` SHA-256 stayed 966208f925453dbbfa43947a1ad8051097569f3c765cba0ed217b48a8dfe54e6 before and after. The CA private key was not copied into any node bundle; PgBouncer remained uninstalled; HAProxy and PostgreSQL/Patroni were unchanged; child gate exit code was 0.

Exact PgBouncer package read-only inspection on u1c-03 - 2026-08-23: PASS. The staged u1c-03 certificate fingerprint remained BC:50:50:8D:FC:60:37:5E:E8:B6:A0:3B:93:41:D7:AB:53:CB:BD:C7:E1:86:3B:1D:FD:AD:D8:94:9E:45:7F:6E and still

verified for `pgbouncer.internal.lorawan.com`. PgBouncer remained absent and port 6432 free. Ubuntu Noble candidate 1.22.0-1build4 matched the pin; apt simulation showed PgBouncer plus its normal dependencies with no removals. The exact `.deb` was downloaded and extracted off-path only. Its systemd unit is `/usr/lib/systemd/system/pgbouncer.service`, with `User=postgres`, no explicit `Group=`, and `ExecStart=/usr/sbin/pgbouncer /etc/pgbouncer/pgbouncer.ini`. The package declares `/etc/pgbouncer/pgbouncer.ini` and `/etc/pgbouncer/userlist.txt` as `conffiles`. Its `postinst` uses Debian/systemd helpers that enable new installations and attempt to start/restart the service, so the install canary must suppress service starts before `apt-get install`; merely planning to stop the service afterward is not enough. The inspection installed nothing, created no listener, and child exit code was 0.

Why this changes the install order: do not guess a pgbouncer Unix account. The exact package runs as host user `postgres`. Because `Group=` is absent, systemd uses the service user's normal group context; the install canary must discover the actual host `postgres` primary group after package dependencies are configured, then install `/etc/lorawan-pki/pgbouncer` as root-owned with only that real service group able to traverse/read the key. Keep the package-default PgBouncer configuration stopped and disabled until the TLS, auth file, logical DNS mapping, and HAProxy upstream settings are validated off-path.

ulc-03 PgBouncer package + local TLS install canary - 2026-08-23: PASS. The gate re-proved Patroni role endpoints HTTP 200, HAProxy active with exact listeners `10.104.0.8:15432` and `10.104.0.8:15433`, and Spilo identity `7e1c213d1694f37aa08bf2a996a64947b207df5ca9c4366ff0debfa8f2bb123|2026-08-22T14:32:18.26248358Z|0`. The staged `ulc-03` certificate fingerprint and key hash matched the issued identity. A temporary `/usr/sbin/policy-rc.d` returned 101, so package configuration did not start `ssl-cert.service`, `postgresql.service`, or `pgbouncer.service`. The exact PgBouncer 1.22.0-1build4 package installed with its dependency set. PgBouncer was observed inactive but package-enabled, then explicitly disabled to inactive/disabled; the temporary package policy was restored/removed afterward.

The installed package created host service identity `uid=110(postgres) gid=114(postgres)`, with supplementary group `ssl-cert` GID 113; systemd still declares `User=postgres` and no explicit `Group=`. `pg_lsclusters` returned no host PostgreSQL cluster. Package-default files were backed up under `/etc/pgbouncer/phase7-package-default-20260823-151910`; observed packaged modes were `0640 postgres:postgres` for `pgbouncer.ini` and `userlist.txt`, and `0644 root:root` for `/etc/default/pgbouncer`. The local TLS directory `/etc/lorawan-pki/pgbouncer` is `0750 root:postgres`; `ca.crt`, `server.crt`, and `server.key` are each `0640 root:postgres`. The `postgres` service identity can read all three while `opsadmin` cannot read `server.key`. Installed certificate fingerprint, chain, shared logical hostname, node hostname, node IP, and key/certificate public-key equality all re-verified successfully. PgBouncer remained inactive/disabled with no `:6432` listener, Spilo identity stayed byte-for-byte unchanged, HAProxy stayed active, and all Patroni role endpoints remained HTTP 200.

ulc-03 PgBouncer dependency-service collateral check - 2026-08-23: PASS. PgBouncer remained inactive/disabled with no `:6432` listener. `postgresql.service` is loaded, inactive, enabled, `Type=oneshot`, and its unit is only the PostgreSQL meta-service with `ExecStart=/bin/true`; `pg_lsclusters` returned none, no `postgresql@*.service` instance was loaded or running, and `/etc/postgresql` contains only the package-created top-level directory. `ssl-cert.service` is also loaded, inactive, enabled, `Type=oneshot`; its only purpose is generating the default snakeoil certificate when `/etc/ssl/private/ssl-cert-snakeoil.key` is absent. The temporary `policy-rc.d` is absent after restoration. PgBouncer TLS permissions remain `0750 root:postgres` for the directory and `0640 root:postgres` for `ca.crt`, `server.crt`, and `server.key`; the service identity can still read the key. HAProxy listeners remain exactly `10.104.0.8:15432` and `10.104.0.8:15433`, and Patroni role endpoints remain HTTP 200 for `ulc-02 /leader`, `ulc-01 /replica`, and `ulc-03 /replica`. Child gate exit code was 0 and the SSH shell remained alive.

ulc-03 PgBouncer dependency-service hygiene - 2026-08-23: PASS. `postgresql.service` and `ssl-cert.service` were disabled and remained inactive; neither package was uninstalled and neither unit was masked. `pg_lsclusters` still returned no host cluster, no running `postgresql@*.service` existed, and the existing `/etc/ssl/private/ssl-cert-snakeoil.key` was only inventoried (`0640 root:ssl-cert`) rather than deleted. PgBouncer stayed inactive/disabled with no `:6432` listener. `/etc/lorawan-pki/pgbouncer` remained `0750 root:postgres` and its `CA/certificate/key` remained `0640 root:postgres`, with the `postgres` service identity still able to read the private key. HAProxy listeners stayed exactly `10.104.0.8:15432` and `10.104.0.8:15433`; all expected

Patroni role endpoints remained HTTP 200; and Spilo identity remained byte-for-byte unchanged at 7e1c213d1694f37aa08bf2a996a64947b207df5ca9c4366ff0debfa8f2bb123|2026-08-22T14:32:18.26248358Z|0. Child gate exit code was 0.

Decision: package-side cleanup is complete on the canary. Recheck these disabled dependency units after future PgBouncer/postgresql-common package upgrades. Keep PgBouncer stopped while the next read-only gate proves postgres-ha.internal resolves to the local HAProxy address on ulc-03 and confirms the four intended application roles have non-null SCRAM-SHA-256 verifiers. Only then create the protected userList.txt and off-path PgBouncer configuration.

7.3 Install and inspect

```
haproxy -vv
pgbouncer --version
systemctl cat haproxy
systemctl cat pgbouncer
```

Use supported distribution packages or pinned artifacts that match the operating-system release. Keep the observed HAProxy and PgBouncer versions with the package source and previous package reference because configuration directives and rollback behavior can differ between releases.

Observed on 2026-08-23: the initial preflight found HAProxy and PgBouncer absent on all three nodes. Ubuntu Noble offers HAProxy 2.8.16-0ubuntu0.24.04.3 from noble-updates / noble-security and PgBouncer 1.22.0-1build4 from noble/universe. HAProxy is installed and live-validated on all three nodes. Three unique PgBouncer server identities for pgbouncer.internal.lorawan.com are issued and verified. PgBouncer is installed only on canary node ulc-03, where it remains stopped/disabled with no :6432 listener and its verified local certificate/key/CA are installed under /etc/lorawan-pki/pgbouncer; ulc-01 and ulc-02 still have no PgBouncer package or installed PgBouncer private key. On ulc-03, the package-created postgresql.service and ssl-cert.service are explicitly inactive/disabled, no host PostgreSQL cluster exists, and no dependency package was removed. Continue with logical-name resolution and protected SCRAM-verifier preflight before writing or starting the PgBouncer configuration.

7.4 HAProxy configuration

Back up /etc/haproxy/haproxy.cfg, then configure private database frontends. Replace IPs and certificate behavior with the approved design.

```
global
    log /dev/log local0
    log /dev/log local1 notice
    user haproxy
    group haproxy
    daemon
    maxconn <HAPROXY_MAX_CONNECTIONS>

    ssl-default-bind-options ssl-min-ver TLSv1.2
    ssl-default-server-options ssl-min-ver TLSv1.2

defaults
    log global
    mode tcp
    option tcplog
    timeout connect 5s
    timeout client 60s
    timeout server 60s
    timeout check 5s

frontend postgres_primary
```

```

bind <THIS_HOST_PRIVATE_IP>:15432
default_backend patroni_primary

backend patroni_primary
    option httpchk GET /primary
    http-check expect status 200
    default-server inter 2s fall 3 rise 2 on-marked-down shutdown-sessions
    server ha-01 <HA01_PRIVATE_IP>:5432 check port 8008
    server ha-02 <HA02_PRIVATE_IP>:5432 check port 8008
    server ha-03 <HA03_PRIVATE_IP>:5432 check port 8008

frontend postgres_replicas
    bind <THIS_HOST_PRIVATE_IP>:15433
    default_backend patroni_replicas

backend patroni_replicas
    balance roundrobin
    option httpchk GET /replica?lag=<APPROVED_MAX_REPLICA_LAG>
    http-check expect status 200
    default-server inter 3s fall 3 rise 2
    server ha-01 <HA01_PRIVATE_IP>:5432 check port 8008
    server ha-02 <HA02_PRIVATE_IP>:5432 check port 8008
    server ha-03 <HA03_PRIVATE_IP>:5432 check port 8008

```

Patroni /primary returns HTTP 200 only for the member that is primary and owns the leader lock. Replica checks can include a maximum lag. Verify endpoint semantics against the pinned Patroni version.

If Patroni REST uses TLS or authentication, configure HAProxy accordingly; do not disable REST protection to simplify health checks.

OpenBao KMS frontend

Add a private TCP frontend on each adapter host:

```

frontend openbao_kms
    mode tcp
    bind <THIS_HOST_PRIVATE_IP>:18200
    default_backend openbao_nodes

backend openbao_nodes
    mode tcp
    balance roundrobin
    option httpchk GET /v1/sys/health?standbyok=true
    http-check expect status 200
    default-server inter 2s fall 3 rise 2

    # Client traffic remains raw TLS pass-through because the server lines do
    # not enable `ssl` for normal proxied traffic. `check-ssl` encrypts only
    # the HAProxy health check.
    server openbao-1 <HA01_PRIVATE_IP>:8200 check check-ssl verify required ca-file /etc/lorawan-
pki/openbao/ca.crt check-sni openbao-kms.internal.lorawan.com
    server openbao-2 <HA02_PRIVATE_IP>:8200 check check-ssl verify required ca-file /etc/lorawan-
pki/openbao/ca.crt check-sni openbao-kms.internal.lorawan.com
    server openbao-3 <HA03_PRIVATE_IP>:8200 check check-ssl verify required ca-file /etc/lorawan-
pki/openbao/ca.crt check-sni openbao-kms.internal.lorawan.com

```

Validate that the pinned HAProxy version supports the shown check-ssl and check-sni syntax before reload. If it differs, adapt only the syntax, not the behavior: the health check must use HTTPS, verify the OpenBao CA/name, call /v1/sys/health?standbyok=true, and accept only an initialized/unsealed usable backend.

Do **not** accept plain TCP-connect-only health as the final KMS test. OpenBao's health endpoint can distinguish initialized/unsealed active or standby nodes from sealed/uninitialized nodes.

Why standby is valid here: an unsealed OpenBao HA standby can forward requests to the active node. A sealed node is not usable redundancy and must not be considered a healthy adapter backend.

7.5 Validate HAProxy before reload

```
sudo haproxy -c -V -f /etc/haproxy/haproxy.cfg
sudo systemctl reload haproxy
sudo systemctl status haproxy --no-pager -l
sudo ss -lntp | grep -E ':(15432|15433|18200|18883)\b'
```

The PostgreSQL HAProxy listeners bind only to each host's private interface. The OpenBao KMS listener exists only on ha-01/02, also private. Live Phase 9 uses private ChirpStack MQTT :18883 on u1c-01/02; Phase 12A later adds the separate private Node-RED MQTT frontend :18884 on u1c-03. None of these private listeners is exposed directly to the Internet.

Check each Patroni endpoint directly from the current host:

```
for host in <DB_01_PRIVATE_IP> <DB_02_PRIVATE_IP> <DB_03_PRIVATE_IP>; do
    printf '%s ' "$host"
    curl -sS -o /dev/null -w '%{http_code}\n' "http://$host:8008/primary"
done
```

Exactly one should return 200. Adapt for HTTPS and client certificates when enabled.

7.6 PostgreSQL routing test

```
psql 'host=postgres-ha.internal hostaddr=<THIS_HOST_PRIVATE_IP> port=15432 dbname=postgres user=
<MONITOR_ROLE> sslmode=verify-full' \
-c "SELECT inet_server_addr(), pg_is_in_recovery();"
```

Expected: pg_is_in_recovery = false.

For the replica frontend:

```
psql 'host=postgres-ha.internal hostaddr=<THIS_HOST_PRIVATE_IP> port=15433 dbname=postgres user=
<MONITOR_ROLE> sslmode=verify-full' \
-c "SELECT inet_server_addr(), pg_is_in_recovery();"
```

Expected: true. Do not send ChirpStack writes to the replica endpoint.

7.7 PgBouncer baseline

Create /etc/pgbouncer/pgbouncer.ini and protect the authentication material.

```
[databases]
chirpstack = host=postgres-ha.internal port=15432 dbname=chirpstack
lorawan_telemetry = host=postgres-ha.internal port=15432 dbname=lorawan_telemetry

[pgbouncer]
listen_addr = <THIS_HOST_PRIVATE_IP>
listen_port = 6432
unix_socket_dir = /run/postgresql

pool_mode = session

# Tiny POC starting limits. These are intentionally small because PostgreSQL
# itself starts with max_connections=40 and only a few sensors are active.
max_client_conn = 50
default_pool_size = 3
min_pool_size = 0
```

```

reserve_pool_size = 1
reserve_pool_timeout = 5
max_db_connections = 8

server_connect_timeout = 5
server_login_retry = 5
server_lifetime = 3600
server_idle_timeout = 300
server_fast_close = 1
client_login_timeout = 30
query_wait_timeout = 15

server_tls_sslmode = verify-full
server_tls_ca_file = /etc/lorawan-pki/pgbouncer/ca.crt

client_tls_sslmode = require
client_tls_cert_file = /etc/lorawan-pki/pgbouncer/server.crt
client_tls_key_file = /etc/lorawan-pki/pgbouncer/server.key

auth_type = scram-sha-256
auth_file = /etc/pgbouncer/userlist.txt

admin_users = <PGBOUNCER_ADMIN_ROLE>
stats_users = <PGBOUNCER_STATS_ROLE>

log_connections = 1
log_disconnections = 1
log_pooler_errors = 1
stats_period = 60

```

Use the PgBouncer bundle's `ca.crt` for `server_tls_ca_file` as well as distributing that CA to clients. It is byte-for-byte the same commissioned internal CA that signed PostgreSQL, but it lives under the host PgBouncer service's own readable PKI directory. Do **not** loosen `/etc/lorawan-pki/postgres` just to let the host postgres account traverse it; that directory's numeric ownership was commissioned for the Spilo/container boundary and maps differently on the host. Keeping the public CA copy in `/etc/lorawan-pki/pgbouncer/ca.crt` preserves both boundaries while `verify-full` still validates `postgres-ha.internal`.

Start with session pooling because it preserves connection-level behavior. Transaction pooling can reduce server connections further but may break applications that rely on session state, temporary tables, advisory locks, certain prepared-statement behavior, or SET persistence. Enable it only after ChirpStack version-specific staging tests.

7.8 Authentication choices

Option A: protected auth file

Use SCRAM verifier strings, not plaintext passwords. Keep `/etc/pgbouncer/userlist.txt` mode 640 or stricter and owned by the PgBouncer service group.

Rotation sequence:

1. change PostgreSQL role secret in a controlled window;
2. update the PgBouncer verifier atomically;
3. run `RELOAD`;
4. reconnect affected pools;
5. verify application recovery;
6. remove old secret from the approved store.

Option B: auth_query

Use a dedicated non-login or tightly restricted authentication role and a reviewed `SECURITY DEFINER` function owned by a trusted administrator. The function must have a fixed `search_path`, expose only username and password verifier, and grant execution only to the PgBouncer auth role.

Do not grant the PgBouncer process general access to `pg_shadow`, `superuser`, or application tables.

7.9 POC pool sizing

Do not size PgBouncer for the future fleet yet. The POC starts with `default_pool_size=3`, `reserve_pool_size=1`, and `max_db_connections=8` on each host while PostgreSQL starts at `max_connections=40`.

Why so small: a few sensors do not need dozens of simultaneous database server sessions. PgBouncer is present to prove the future connection-pooling boundary, not to generate artificial connection load.

Watch `SHOW POOLS` during failover. If clients wait under the actual POC workload, increase the pool gradually and record the reason. Do not raise PostgreSQL and PgBouncer limits preemptively.

7.10 Start and validate PgBouncer

```
sudo pgbouncer -t /etc/pgbouncer/pgbouncer.ini 2>/dev/null || true
sudo systemctl restart pgbouncer
sudo systemctl status pgbouncer --no-pager -l
sudo ss -lntp | grep ':6432'
```

Use the validation option supported by the installed version; inspect `pgbouncer --help` first.

Test both logical databases through the local pool:

Use the shared certificate name for hostname verification while forcing the connection to this host's private IP:

```
psql 'host=pgbouncer.internal.lorawan.com hostaddr=<THIS_HOST_PRIVATE_IP> port=6432 dbname=chirpstack
user=chirpstack sslmode=verify-full sslrootcert=/etc/lorawan-pki/pgbouncer/ca.crt' \
-c "SELECT current_database(), inet_server_addr(), pg_is_in_recovery();"

psql 'host=pgbouncer.internal.lorawan.com hostaddr=<THIS_HOST_PRIVATE_IP> port=6432 dbname=lorawan_telemetry
user=telemetry_reader sslmode=verify-full sslrootcert=/etc/lorawan-pki/pgbouncer/ca.crt' \
-c "SELECT current_database(), inet_server_addr(), pg_is_in_recovery();"
```

Run the tests appropriate to each host: ChirpStack on `ha-01/02`; telemetry clients on `ha-03`; adapter outbox access on `ha-01/02`. PgBouncer stays on the host's restricted private interface. Its server connection through HAProxy to PostgreSQL must use verified TLS.

7.11 Administration and metrics

Connect to the virtual pgbouncer database with the `stats/admin` role:

```
SHOW VERSION;
SHOW CONFIG;
SHOW DATABASES;
SHOW POOLS;
SHOW CLIENTS;
SHOW SERVERS;
SHOW STATS;
```

Alert when:

- `cl_waiting` remains above zero;
- `maxwait` increases;
- server login failures occur;
- pools approach `max_db_connections`;
- frequent server disconnects follow database failover;
- file descriptors or memory approach limits.

7.12 Planned database maintenance

For a controlled Patroni switchover:

1. confirm PgBouncer is healthy on all three hosts;
2. observe `SHOW POOLS`;
3. perform Patroni switchover;
4. allow each local HAProxy to select the new primary;
5. issue `RECONNECT chirpstack`; or `RECONNECT lorawan_telemetry`; only if old server connections remain;
6. verify a ChirpStack DB query, one Node-RED telemetry insert, and one Grafana read through the unchanged endpoints;
7. when the reviewed Fabric adapter is deployed, also verify one adapter outbox query; otherwise verify the outbox directly and record adapter execution as BLOCKED.

7.13 Failover validation

During a staging primary failure:

- HAProxy must mark the old primary down;
- one replica must be promoted by Patroni;
- HAProxy must route new connections only to the promoted primary;
- ChirpStack must reconnect through HAProxy; if PgBouncer is enabled, it must discard or replace broken server connections;
- ChirpStack must retry and recover without manual DSN changes;
- the test transaction sequence must match the selected RPO mode.

7.14 Final checks

- The private PostgreSQL HAProxy frontend and PgBouncer validate on all three hosts.
- Public ChirpStack/MQTT HAProxy routing remains only on `ha-01/02`, with public listeners bound to each host's anchor IP and reached through the single Reserved IPv4.
- Exactly one PostgreSQL primary is routed by every local 15432 frontend.
- Both `chirpstack` and `lorawan_telemetry` exist in PgBouncer.
- PgBouncer uses the intentionally small POC limits and shows no sustained client wait under the few-sensor workload.
- Patroni changes recover without editing ChirpStack, Node-RED, or Grafana database endpoints; when Fabric adapters are deployed, their endpoint remains unchanged too.

7.15 Phase 7 validation progress record

Current POC validation progress:

Completed:

- PgBouncer TLS listener validation on ulc-01.
- PgBouncer package, TLS, SCRAM, configuration, and runtime validation completed on ulc-02.
- SCRAM authentication validation for:
 - chirpstack.
 - fabric_adapter.
 - telemetry_reader.
 - telemetry_writer.
- Verified PgBouncer routes successful sessions through the PostgreSQL HAProxy primary endpoint.
- Verified returned backend state is the writable PostgreSQL leader (`pg_is_in_recovery() = false`).
- Verified backend TLS encryption from PgBouncer to PostgreSQL using TLS 1.3.
- Verified PgBouncer idle timeout behavior after the 75-second regression test without connection failure.
- Verified HAProxy 360-second PostgreSQL timeout ordering remains above PgBouncer `server_idle_timeout = 300s`.

Troubleshooting note:

- PgBouncer admin console access is separate from application database users.
- A failed `SHOW POOLS` login does not indicate PostgreSQL failure. It only means the PgBouncer admin identity is not configured or the supplied admin credential is invalid.
- When generating or checking PgBouncer administrative access, perform PostgreSQL role operations against the current Patroni leader only. Running `CREATE ROLE` on a replica will fail with `cannot execute CREATE ROLE in a read-only transaction`.

Next validation:

1. Configure or verify the PgBouncer admin/stats identities.
2. Run `SHOW POOLS`, `SHOW SERVERS`, and `SHOW STATS`.
3. Perform session reuse and failover connection validation before enabling PgBouncer at boot.

Latest ULC-02 commissioning evidence:

- PgBouncer service enabled for boot startup.
- PgBouncer active state verified after enabling.
- Private listener confirmed on `10.104.0.4:6432`.
- Client TLS certificate validation succeeded using the PgBouncer certificate identity.
- TLS negotiation verified with TLS 1.3 and `TLS_AES_256_GCM_SHA384`.
- Application database connectivity validated through PgBouncer for:
 - chirpstack.
 - fabric_adapter.
 - telemetry_reader.
 - telemetry_writer.
- PgBouncer backend TLS connections successfully established to PostgreSQL through local HAProxy.
- The observed server `conn crashed? (age=60s)` messages occurred during the idle timeout regression period and were followed by successful reconnections; the 75-second idle regression test completed

successfully.

7.16 PgBouncer administration console validation

The PgBouncer admin console is not enabled by default. `admin_users` and `stats_users` must be explicitly configured before running `SHOW POOLS`, `SHOW SERVERS`, or `SHOW STATS`.

Application database users and PgBouncer administration users are separate identities.

Before enabling administrative access:

1. Decide the dedicated PgBouncer admin identity.
2. Configure `admin_users` and/or `stats_users` in `pgbouncer.ini`.
3. Reload PgBouncer.
4. Validate only the intended administrative commands.

Do not use PostgreSQL application users as a substitute for PgBouncer admin access.

7.17 Multi-node PgBouncer commissioning record

The production-style POC deployment uses one PgBouncer instance on each PostgreSQL HAProxy node.

Commissioning status:

Node	Private IP	PgBouncer	TLS	SCRAM auth	Backend test
ulc-01	10.104.0.2	commissioned	verified	verified	passed
ulc-02	10.104.0.4	commissioned	verified	verified	passed
ulc-03	10.104.0.8	commissioned	verified	verified	passed

For each node:

1. Install the pinned PgBouncer package.
2. Keep the service stopped until TLS, authentication, and configuration are installed.
3. Install the node certificate, private key, and CA bundle under `/etc/lorawan-pki/pgbouncer`.
4. Generate `/etc/pgbouncer/userlist.txt` from PostgreSQL SCRAM verifiers.

Use a direct `docker compose exec spilo psql` extraction when generating the file. Avoid deeply nested shell quoting around SQL because it can silently produce an empty `userlist`. Validate that exactly four SCRAM entries exist before installation: `- chirpstack - fabric_adapter - telemetry_reader - telemetry_writer`

5. Install the local PgBouncer configuration using the node private IP for `listen_addr`.
6. Validate TLS before opening the listener.
7. Test application database login through 6432.
8. Enable the service only after successful validation.

The ULC-03 foundation stage originally completed with the package installed but intentionally stopped while its TLS/SCRAM configuration was prepared. That historical checkpoint is superseded by section 7.21: ULC-03 was subsequently commissioned with the same validated TLS, SCRAM, backend-routing, and enablement sequence used by ULC-01 and ULC-02.

7.18 Phase 7 ULC-03 TLS deployment record

ULC-03 PgBouncer TLS material installation was validated before PgBouncer configuration.

Validation completed:

- Source certificate bundle located from the approved issuance directory.
- /etc/lorawan-pki/pgbouncer created with ownership root:postgres.
- CA certificate installed.
- PgBouncer server certificate installed.
- PgBouncer private key installed.
- TLS files protected with mode 640.
- PostgreSQL service account access verified.
- Certificate chain verification passed.
- Certificate hostname verification passed for pgbouncer.internal.lorawan.com.
- Certificate public key and private key public key hashes matched.

7.19 Phase 7 ULC-03 PgBouncer SCRAM and configuration workflow

ULC-03 follows the same protected authentication workflow validated on ULC-01 and ULC-02.

Operational commands must be recorded with the deployment procedure because PgBouncer authentication depends on PostgreSQL SCRAM verifier extraction, not plaintext passwords.

SCRAM extraction workflow:

```
sudo docker compose \\  
-f /etc/lorawan-cloud/spilo/compose.yml \\  
exec -T spilo \\  
psql \\  
-X \\  
-U postgres \\  
-d postgres \\  
-A \\  
-t \\  
-v ON_ERROR_STOP=1 \\  
-c "  
SELECT  
  '\"' || rolname || '\" \"' || rolpassword || '\"'  
FROM pg_authid  
WHERE rolname IN (  
  'chirpstack',  
  'fabric_adapter',  
  'telemetry_reader',  
  'telemetry_writer'  
)  
AND rolcanlogin  
AND rolpassword LIKE 'SCRAM-SHA-256\\$%'  
ORDER BY rolname;  
" |  
sudo tee /run/pgbouncer-userlist.final >/dev/null
```

Install the generated verifier file:

```
sudo install \\  
-m 640 \\  
-o root \\  
-g postgres \\  
/run/pgbouncer-userlist.final /etc/pgbouncer/
```

```
/run/pgbouncer-userlist.final \\
/etc/pgbouncer/userlist.txt
```

Install and verify the node-specific PgBouncer configuration:

```
sudo install \
-m 640 \
-o root \
-g postgres \
/tmp/pgbouncer-ulc03.ini \
/etc/pgbouncer/pgbouncer.ini

sudo grep -E \
'^(\listen_addr|listen_port|pool_mode|max_client_conn|default_pool_size|max_db_connections|server_idle_timeout
|server_tls_sslmode|server_tls_ca_file|client_tls_sslmode|client_tls_cert_file|client_tls_key_file|auth_type|
auth_file)[[:space:]]*=' \
/etc/pgbouncer/pgbouncer.ini
```

Validation requirements:

- Exactly four SCRAM entries must exist.
- Entries must be `chirpstack`, `fabric_adapter`, `telemetry_reader`, and `telemetry_writer`.
- `/etc/pgbouncer/userlist.txt` must remain unreadable by `opsadmin`.
- PostgreSQL service account must be able to read the file.

The same installation boundary is used for ULC-01, ULC-02, and ULC-03 before enabling PgBouncer services.

7.20 Phase 7 ULC-03 pre-start validation record

ULC-03 PgBouncer configuration reached the pre-start validation boundary.

Validated:

- PgBouncer configuration installed with node-local listener:
- `listen_addr` = `10.104.0.8`
- `listen_port` = `6432`
- TLS material:
- CA certificate readable by PostgreSQL service account.
- Server certificate readable by PostgreSQL service account.
- Private key readable by PostgreSQL service account.
- Certificate chain verification passed.
- SCRAM authentication file:
- `/etc/pgbouncer/userlist.txt` installed.
- Four SCRAM verifier entries present.
- File ownership and permissions validated.
- HAProxy PostgreSQL endpoints available:
- `10.104.0.8:15432`
- `10.104.0.8:15433`
- Local resolver mapping validated:
- `postgres-ha.internal -> 10.104.0.8`

Historical checkpoint: these were the next ULC-03 steps at the time this pre-start record was captured. They were subsequently completed successfully. Section 7.21 contains the authoritative final three-node state.

7.21 Phase 7 Complete PgBouncer HA Commissioning Record

Objective

Deploy PgBouncer on all PostgreSQL HA nodes as the stable client connection layer while keeping Patroni responsible for PostgreSQL leader election and HAProxy responsible for primary routing.

Final architecture:

```
Application clients
  |
  v
PgBouncer :6432
  |
  v
HAProxy :15432
  |
  v
Patroni PostgreSQL primary
  |
  v
Current leader
```

PgBouncer does not replace Patroni or HAProxy. It provides connection pooling and TLS termination for database clients while HAProxy continues selecting the writable PostgreSQL leader.

Phase 7 Execution Record

Step 1 - PgBouncer package installation

Performed on:

- ULC-01
- ULC-02
- ULC-03

Validation:

```
pgbouncer --version
systemctl is-active pgbouncer
systemctl is-enabled pgbouncer
ss -H -lnt | grep ':6432'
```

Initial safety requirement:

```
PgBouncer installed
PgBouncer stopped
PgBouncer disabled
No :6432 listener exposed
```

The service remained disabled until TLS, authentication, and configuration validation completed.

Step 2 - TLS deployment

TLS materials were installed separately on each node:

```
/etc/lorawan-pki/pgbouncer/  
├── ca.crt  
├── server.crt  
└── server.key
```

Permissions:

```
Directory:  
750 root:postgres  
  
Files:  
640 root:postgres
```

Validation performed:

```
openssl verify \  
-CAfile /etc/lorawan-pki/pgbouncer/ca.crt \  
-verify_hostname pgbouncer.internal.lorawan.com \  
/etc/lorawan-pki/pgbouncer/server.crt
```

Required result:

```
server.crt: OK
```

Certificate and private key matching was verified by comparing public key hashes.

Step 3 - SCRAM authentication installation

PgBouncer uses PostgreSQL SCRAM verifiers.

Plaintext database passwords are not stored by PgBouncer.

The userlist contains:

```
chirpstack  
fabric_adapter  
telemetry_reader  
telemetry_writer
```

Generated file:

```
/etc/pgbouncer/userlist.txt
```

Permissions:

```
640 root:postgres
```

Validation:

```
sudo wc -l /etc/pgbouncer/userlist.txt
```

Expected:

```
4
```

Required access model:

```
postgres user    -> read allowed
opsadmin user    -> read denied
```

Step 4 - PgBouncer configuration

Each node uses its own private IP.

Configuration pattern:

```
listen_addr = <NODE_PRIVATE_IP>
listen_port = 6432

pool_mode = session

max_client_conn = 50
default_pool_size = 3
max_db_connections = 8

server_tls_sslmode = verify-full
server_tls_ca_file = /etc/lorawan-pki/pgbouncer/ca.crt

client_tls_sslmode = require
client_tls_cert_file = /etc/lorawan-pki/pgbouncer/server.crt
client_tls_key_file = /etc/lorawan-pki/pgbouncer/server.key

auth_type = scram-sha-256
auth_file = /etc/pgbouncer/userlist.txt
```

Databases exposed:

```
chirpstack
lorawan_telemetry
```

Step 5 - Runtime TLS validation

Every node passed:

```
openssl s_client \
-starttls postgres \
-connect <NODE_IP>:6432 \
-CAfile /etc/lorawan-pki/pgbouncer/ca.crt \
-verify_hostname pgbouncer.internal.lorawan.com
```

Verified:

```
TLSv1.3
TLS_AES_256_GCM_SHA384
Certificate verification OK
```

Step 6 - Application database validation

Validated users:

```
chirpstack
fabric_adapter
telemetry_reader
telemetry_writer
```

Validation query:

```
SELECT
  current_user,
  inet_server_addr(),
  pg_is_in_recovery();
```

Expected:

```
pg_is_in_recovery = false
```

Successful routing path:

```
PgBouncer
  |
  v
HAProxy :15432
  |
  v
Patroni leader PostgreSQL
```

Phase 7 Failover Validation

Controlled Patroni switchover

Initial state:

```
ulc-02
10.104.0.4
Leader
```

Switchover command:

```
Primary: ulc-02
Candidate: ulc-01
Time: now
```

Result:

```
ulc-01
10.104.0.2
Leader

ulc-02
10.104.0.4
Replica

ulc-03
10.104.0.8
Replica
```

Validation:

```
curl -s http://10.104.0.2:8008/
curl -s http://10.104.0.4:8008/
curl -s http://10.104.0.8:8008/
```

Confirmed:

```
Exactly one primary exists.
Replication lag = 0.
```

PgBouncer Failover Routing Test

After Patroni promotion:

Client connection through PgBouncer returned:

```
current_user = chripstack
inet_server_addr = 10.104.0.2
pg_is_in_recovery = false
```

Meaning:

```
Application
|
v
PgBouncer
|
v
HAProxy
|
v
New Patroni leader
|
v
ulc-01 PostgreSQL
```

No application connection string changes were required.

No PgBouncer restart was required.

No PostgreSQL restart was required.

Final Phase 7 State

Node	IP	PgBouncer	TLS	SCRAM	HA Routing
ULC-01	10.104.0.2	Active + Enabled	PASS	PASS	PASS
ULC-02	10.104.0.4	Active + Enabled	PASS	PASS	PASS
ULC-03	10.104.0.8	Active + Enabled	PASS	PASS	PASS

Final verified capabilities:

- PgBouncer deployed on all PostgreSQL HA nodes.
- TLS encryption enabled between clients and PgBouncer.
- TLS encryption enabled between PgBouncer and PostgreSQL.
- SCRAM authentication enabled.
- HAProxy PostgreSQL routing preserved.
- Patroni leader changes handled successfully.
- Database clients continued operating after failover.

Phase 7 status:

COMPLETE

Next: [08-mqtt-and-valkey.md](#)

Canonical source: deployment/technology-guides/08-haproxy-pgbouncer.md. For any mutable production-status statement, check the dated baseline and verified live state.